

On-Chain Diploma Verification on Ethereum: Cost Trade-Offs of Per-Record Storage and Merkle-Batch Architectures

Nazar Shcherbina

Faculty of Electrical Engineering
Warsaw University of Technology, Warsaw, Poland
nazar.shcherbina.stud@pw.edu.pl

Bartosz Sawicki

Faculty of Electrical Engineering
Warsaw University of Technology, Warsaw, Poland

Abstract—Blockchain can make academic credentials independently verifiable, yet on Ethereum the cost of persistent storage threatens practicality. This paper quantifies that cost in *UniVerify*, a prototype diploma-verification system, by comparing two on-chain representations: *packed per-record storage*, where each diploma occupies one slot, and *Merkle batch issuance*, where an entire graduation cohort is represented by a single root.

Four contract variants are benchmarked—**Packed**, **Baseline**, **OpenZeppelin**, and **Optimized**—across proof depths $d \in \{0, 1, 4, 8, 10, 12\}$. Merkle batching reduces per-diploma issuance cost by 98.9% at batch size $N=100$, shifting part of the cost to proof-based verification. **Optimized** lowers verification gas by 6.6% versus **Baseline** and by 3.7% versus **OpenZeppelin**. A closed-form gas-savings model is derived and validated at six proof depths with residuals below 0.2%. Under mainnet conditions (40 gwei, 1 ETH=\$2,500), issuing 1,000 diplomas costs \$4,820 per-record versus \$5.09 with batching. The results show that architecture dominates micro-optimization for institutional diploma issuance.

Index Terms—Ethereum, Solidity, gas optimization, Merkle tree, smart contracts, diploma verification

I. INTRODUCTION

The global market for fraudulent academic credentials is estimated to exceed one billion dollars annually [1], yet cross-border diploma verification still relies on slow, manual contact with the issuing university. Blockchain-based registries offer an alternative: once a credential commitment is published on-chain, any party can validate it independently, without relying on a single institutional database.

The main challenge is economic. On Ethereum, each persistent storage write costs roughly 20,000 gas. If every diploma is stored individually, institutions issuing hundreds or thousands of diplomas per cycle face prohibitive costs. The key question is therefore not *whether* blockchain can be used, but *how credentials should be represented on-chain so that issuance remains practical*.

This paper studies that question through *UniVerify*, a prototype diploma-verification system. Two architectures are compared:

- 1) **Per-record storage** — each diploma is stored in its own mapping slot, verified in constant time.

- 2) **Merkle batch issuance** — one Merkle root represents an entire cohort; each diploma is verified via an off-chain proof.

The contribution is practical rather than conceptual: this work quantifies the gas implications of deployable Solidity designs and derives a closed-form model predicting optimization savings across proof depths.

A. *UniVerify* System

UniVerify is a prototype comprising a Solidity smart contract for on-chain storage and verification, and an off-chain TypeScript SDK for Merkle tree construction and proof generation. Universities act as *issuers* who publish diploma commitments on-chain; a verifier checks a diploma using the document hash and, in the Merkle variant, a proof of inclusion in the published batch root.

B. Contributions

- 1) A quantitative comparison of packed per-record storage and Merkle-batch issuance across deployment, issuance, verification, and revocation cost.
- 2) A four-way benchmark of **Packed**, **Baseline**, **OpenZeppelin** [9], and **Optimized**, separating architectural, library, and EVM-level effects.
- 3) A closed-form gas-savings model validated at six proof depths with residuals below 0.2%.
- 4) An economic analysis translating gas savings into institutional issuance cost.

II. RELATED WORK

Grech and Camilleri [2] identified credential verification as a primary educational blockchain application. Lizcano et al. [4] proposed a decentralized credential framework on IPFS and Ethereum but omitted low-level cost analysis. Alketbi et al. [3] surveyed broader blockchain use in public-sector services, including education records. More recently, the W3C Verifiable Credentials standard [10] defines a data model for tamper-evident claims but is agnostic to cost; Arndt et al. [11] benchmarked on-chain credential anchoring on

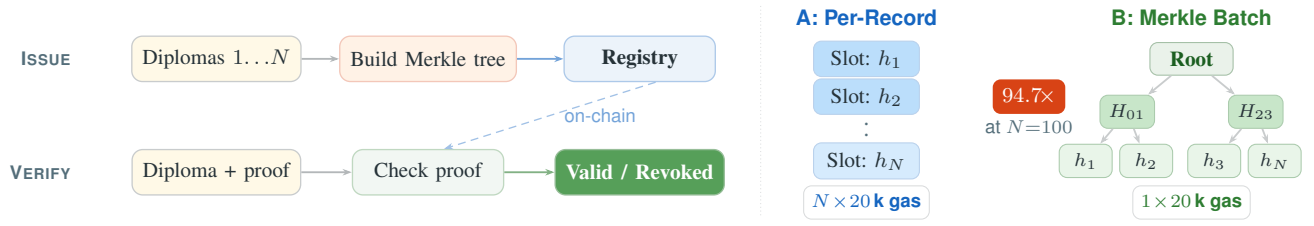


Fig. 1: UniVerify system architecture. *Left*: issuance and verification workflow (off-chain view calls cost 0 gas). *Right*: on-chain cost comparison of the two issuance models.

Ethereum L2 chains without comparing Merkle batching to per-record storage.

Merkle trees [5] provide the standard mechanism for compact set-membership proofs. OpenZeppelin’s MerkleProof [9] is the de facto production Solidity library. Alternative structures—sparse Merkle trees, Verkle trees [12], RSA accumulators—remain less practical for ordinary EVM contracts.

Ethereum execution costs follow the EVM gas schedule [8]; EIP-2929 further raised cold state-access costs [6]. Most prior work on blockchain credentials focuses on architecture, privacy, or governance. By contrast, this paper isolates a narrower question: how specific Solidity design choices affect issuance and verification cost in a deployable credential registry.

III. SYSTEM DESIGN AND METHODOLOGY

A. Two Verification Architectures

Fig. 1 illustrates the two approaches.

Architecture A — Per-Record Storage. Each diploma is stored in a mapping (`bytes32  $\Rightarrow$  uint256`). The issuer address (160 bits) and a revocation flag (1 bit) are packed into one 256-bit slot. Issuance requires one `SSTORE` per diploma; verification is a single `SLOAD` plus bit masking, yielding constant read cost. This contract is intentionally minimal and serves as a gas baseline.

Architecture B — Merkle Batch. All diplomas in a cohort form a Merkle tree off-chain. Only the 32-byte batch root is stored on-chain via `issueBatchRoot`, so one state write covers the entire batch. Verification requires the caller to supply $d = \lceil \log_2 N \rceil$ sibling hashes. Each leaf is domain-separated: `keccak256(contract||chainId||issuer||batchId||docHash)`, preventing cross-contract and cross-chain replay.

B. Contract Variants

To separate architectural from implementation effects, four contract variants were implemented (Table I).

C. Optimization Techniques

The **Optimized** variant targets two internal functions: `_processProof()` and `_merkleLeaf()`.

T1 — Unchecked loop. `unchecked{++i;}` removes overflow-check overhead.

T2 — Scratch-space hashing. The two 32-byte inputs are written to EVM scratch space and hashed in place, avoiding `abi.encodePacked` memory allocation.

TABLE I: Contract Variants and Key Design Choices

Label	Arch.	Key design choice
Packed	Per-rec.	Packed slot: issuer (160 b) + revoked (1 b) in one <code>uint256</code>
Baseline	Merkle	Hand-written proof loop; leaf via <code>abi.encodePacked</code>
OpenZeppelin	Merkle	OZ <code>MerkleProof.verifyCalldata</code>
Optimized	Merkle	Four EVM-level optimizations (T1–T4) on the hot path

TABLE II: Per-Diploma Issuance Cost

N	Merkle (gas)	Per-record (gas)	Ratio
1	50,939	48,200	0.95 $\times$
10	5,094	48,200	9.5 $\times$
50	1,019	48,200	47 $\times$
100	509	48,200	94.7 $\times$
1,000	51	48,200	945 $\times$

T3 — Validation splitting. The public `merkleLeaf()` checks inputs; a private `_merkleLeaf()` is used internally.

T4 — Assembly preimage. The 112-byte leaf preimage is packed via `mstore/shl` instead of `abi.encodePacked`.

D. Experimental Setup

All measurements use Foundry [7] (Solidity 0.8.20, optimizer at 200 runs). Merkle trees are built on-chain only in the test harness; in deployment they are constructed off-chain. Proof depths $d \in \{0, 1, 4, 8, 10, 12\}$ correspond to batch sizes 2^0 to 2^{12} (1–4,096 diplomas). Gas figures are function-level values from `forge test --gas-report`.

IV. RESULTS

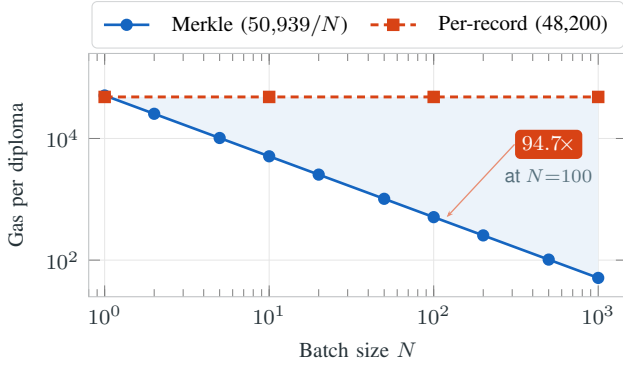
A. Issuance Cost

Table II compares per-diploma issuance cost. Publishing one Merkle batch root costs 50,939 gas regardless of batch size; the amortized cost is $50,939/N$. Per-record issuance costs 48,200 gas per diploma.

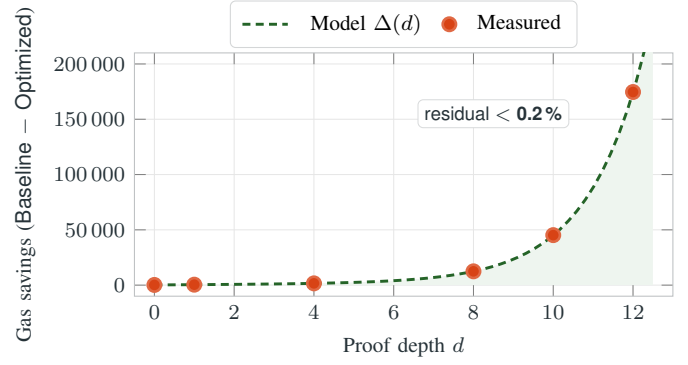
At $N=1$, batching is slightly more expensive than a single direct write. From $N \geq 2$, amortization dominates:

$$\text{gasMerkle}(N) = \frac{50,939}{N}, \quad \text{gasPacked} = 48,200 \quad (1)$$

At $N=100$, Merkle issuance lowers the per-diploma cost by 98.9%.



(a) Per-diploma issuance cost vs. batch size.



(b) Model (3) vs. measured optimization savings.

Fig. 2: (a) Issuance-cost amortization: the shaded region shows savings of Merkle batching over per-record storage. (b) Closed-form model validation with residual annotations.

TABLE III: Verification Gas across Variants

Variant	Min	Avg	Max	vs.Pkd	Δ avg
Packed	2,720	(constant)	—	—	—
Optimized	10,417	11,104	12,103	4.1×	−6.7%
OpenZeppelin	10,560	11,532	12,936	4.2×	−3.1%
Baseline	10,561	11,896	13,841	4.4×	ref.

TABLE IV: Deployment and State-Changing Costs (gas)

Variant	Deploy	Revoke (avg)	Issue
Packed	393,596	29,302	48,200
Optimized	1,018,273	58,303	50,939
OpenZeppelin	1,036,010	56,871	50,939
Baseline	1,142,020	59,610	51,012

B. Verification Cost

Merkle batching shifts part of the cost from issuance to verification. Table III summarizes verification gas for depths $d \in \{0, 1, 4, 8\}$.

Table III reports depths $d \in \{0, 1, 4, 8\}$; the extended range $d \in \{10, 12\}$ appears in Table V, where the **Baseline**–**Optimized** difference is the quantity of interest. Per-record verification costs only 2,720 gas—a single storage read plus bit manipulation. This is the central trade-off: *per-record storage is expensive to write but cheap to read; Merkle batching is cheap to write but costlier to prove*.

Among Merkle variants, **Optimized** achieves the lowest average gas, reducing cost by 6.7% versus **Baseline** and 3.7% versus **OpenZeppelin**.

C. Deployment and Revocation

Packed deploys at roughly one-third the gas of the Merkle variants because its logic is simpler. However, deployment is a one-time cost, whereas issuance recurs every graduation cycle—making repeated issuance the more important metric.

TABLE V: Closed-Form Model Validation (T1–T4 Savings)

d	Leaves	Model Δ	Measured	Residual	Match
0	1	180	180	0	✓
1	2	421	421	0	✓
4	16	1,606	1,606	0	✓
8	256	12,484	12,484	0	✓
10	1,024	45,136	45,218	82 (0.18%)	≈
12	4,096	174,558	174,641	83 (0.05%)	≈

D. Economic Interpretation

With gas price $g=40$ gwei and $p=\$2,500/\text{ETH}$:

$$\text{USD} = \text{gas} \cdot g \cdot p \cdot 10^{-9} \quad (2)$$

This yields \$4.82 per diploma for **Packed** issuance and \$5.09 for one Merkle batch root. At cohort scale the difference is dramatic: 100 diplomas cost \$482 per-record versus \$5.09 batched; 1,000 diplomas cost \$4,820 versus \$5.09. These figures are sensitive to network conditions: at 10 gwei / \$1,500 (bear market) the 1,000-diploma per-record cost drops to \$723, while at 80 gwei / \$4,000 (peak congestion) it rises to \$15,424. The Merkle batch cost stays below \$17 in all scenarios, confirming that the architectural advantage holds across a wide range of market conditions.

E. Closed-Form Savings Model

The gas savings between **Baseline** and **Optimized** decompose into contributions from the four techniques. Let d denote proof depth:

$$\Delta(d) = \underbrace{2^d \cdot 42}_{T4} + \underbrace{96}_{T3} + \underbrace{d \cdot 199}_{T1+T2} + \underbrace{42}_{\text{base}} \quad (3)$$

The constant 42 (“base”) accounts for the fixed-cost savings in leaf construction that are independent of proof depth. Table V validates the model against measured gas differences via `revokeFromBatch`, which exercises the same proof-verification hot path as `statusWithProof`.

For $d \leq 8$ the prediction is exact; at $d \in \{10, 12\}$ the residual remains below 0.2%.

V. DISCUSSION AND CONCLUSION

A. Engineering Trade-off

The dominant design decision is architectural. Packed minimizes verification cost but scales linearly with the number of issued diplomas. Merkle batching inverts this relationship: issuance becomes nearly constant at batch level, at the expense of proof-based verification. For diploma registries this trade-off is favorable—issuance is a paid institutional operation, while verification is a read-only action repeated over the credential lifetime.

B. Optimization Hierarchy

Three layers of optimization emerge:

- 1) **Architecture:** switching to Merkle batching reduces issuance cost by up to 98.9% per diploma.
- 2) **Code-level tuning:** Optimized reduces verification gas by 6.6% and deployment gas by 10.8% vs. Baseline.
- 3) **Library choice:** OpenZeppelin reduces verification gas by 3.1% vs. Baseline.

Micro-optimizations yield measurable gains but cannot compensate for an inefficient storage model. The largest improvement comes from eliminating per-record state writes.

C. Practical Implications

For small-scale or infrequent issuance, per-record storage remains attractive due to its simple logic and constant-time lookup. For cohort-scale institutional use, Merkle batching is the practical choice.

OpenZeppelin's `MerkleProof` is a sensible production default—it captures most available gains while preserving readability and auditability. The additional savings of `Optimized` justify the complexity only when proof verification is a critical path.

D. Limitations and Future Work

This study is limited to gas efficiency on Ethereum L1; it does not address privacy, governance, or interoperability. All gas measurements use Foundry's local EVM, where storage slots are warm after first access within a transaction; on mainnet, EIP-2929 cold-access surcharges (2,100 gas per first SLOAD) would increase absolute verification cost but preserve the relative rankings among variants. The packed per-record contract is a simplified baseline, so the comparison should be read as a cost study rather than a feature-identical system evaluation.

Future work includes benchmarking on Layer 2 networks, studying calldata-sensitive designs under EIP-4844, and evaluating Verkle-based verification once production EVM support is available.

E. Conclusion

This paper compared two deployable architectures for on-chain diploma verification and derived a closed-form model predicting optimization gains across proof depths. The central finding is that *architecture dominates micro-optimization*: institutions issuing more than a handful of diplomas per

cycle should adopt Merkle batching, which reduces per-diploma cost by two orders of magnitude. For the verification path, OpenZeppelin's `MerkleProof` offers the best trade-off between gas savings and code auditability; hand-tuned assembly (`Optimized`) is justified only under extreme throughput requirements. As credential anchoring moves to Layer 2 networks and EIP-4844 reshapes calldata economics, the batching advantage will only grow—making Merkle-based registries the robust default for blockchain-backed academic credentials.

REFERENCES

- [1] A. Ezell and J. Bear, "Degree Mills: The Billion-Dollar Industry That Has Sold Over a Million Fake Diplomas," Prometheus Books, 2012.
- [2] A. Grech and A. F. Camilleri, "Blockchain in Education," *JRC Science for Policy Report*, European Commission, 2017.
- [3] A. Alketbi, Q. Nasir, and M. A. Talib, "Blockchain for Government Services — Use Cases, Security Benefits and Challenges," in *Proc. 15th Learn. Technol. Conf.*, 2018.
- [4] D. Lizcano, J. A. Lara, B. White, and S. Aljawarneh, "Decentralised technology for educational credentials," *J. Comput. High. Educ.*, vol. 32, pp. 9–49, 2020.
- [5] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology — CRYPTO '87*, LNCS, vol. 293, pp. 369–378, 1988.
- [6] V. Buterin et al., "EIP-2929: Gas Cost Increases for State Access Opcodes," *Ethereum Improvement Proposals*, 2020.
- [7] Paradigm, "Foundry: Ethereum Application Development Toolkit," <https://github.com/foundry-rs/foundry>, 2023.
- [8] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," *Ethereum Yellow Paper*, rev. 2024.
- [9] OpenZeppelin, "OpenZeppelin Contracts — MerkleProof," <https://github.com/OpenZeppelin/openzeppelin-contracts>, 2026.
- [10] M. Sporny, D. Longley, and D. Chadwick, "Verifiable Credentials Data Model v2.0," *W3C Recommendation*, 2024.
- [11] T. Arndt, F. Bünger, and U. Fiedler, "On-Chain Credential Anchoring on Ethereum Layer-2 Networks: A Cost and Latency Analysis," in *Proc. IEEE Blockchain*, 2023, pp. 112–119.
- [12] J. Kuszmaul, "Verkle Trees," 2018; V. Buterin et al., "EIP-6800: Ethereum state using a unified Verkle tree," 2023.